

HelixVPN — Phase 0 Spike Plan

HelixVPN — Phase 0 Spike Plan

Revision: 1 **Last modified:** 2026-07-04T12:00:00Z **Note:** This is the original Claude-authored Phase-0 plan that seeded `final/06-phase0-spike-wbs.md`. It is kept as historical/primary-source reference ([04_P0] citations throughout the `final/` set point here); `06-phase0-spike-wbs.md` is the authoritative, independently-verified, DB-ready expansion (3-tier epic→task→subtask, concrete evidence methodology, risk register cross-referenced to `v00-meta/decision-register.md`) — where the two differ in level of detail, the `final/` doc wins per `SPECIFICATION.md`'s spine-authority rule. No content contradictions were found between this document and `06-phase0-spike-wbs.md` during independent gap-analysis (2026-07-04).

Companion to: `HelixVPN-Architecture-Refined.md` **Purpose of Phase 0:** build the thinnest possible *end-to-end* slice that exercises every hard part of the architecture, so that the expensive decisions (iOS viability, edge language, MASQUE-in-Rust maturity, the FFI boundary) are made on **measured evidence**, not on slideware. Phase 0 produces throwaway-quality code on production-quality interfaces: the *traits*, *FFI signatures*, and *wire contracts* defined here are meant to survive into Phase 1; the implementations behind them are allowed to be ugly.

Phase 0 is **time-boxed to ~3-4 focused weeks**. If a gate can't be passed in that window, that is the finding — escalate the decision rather than grind.

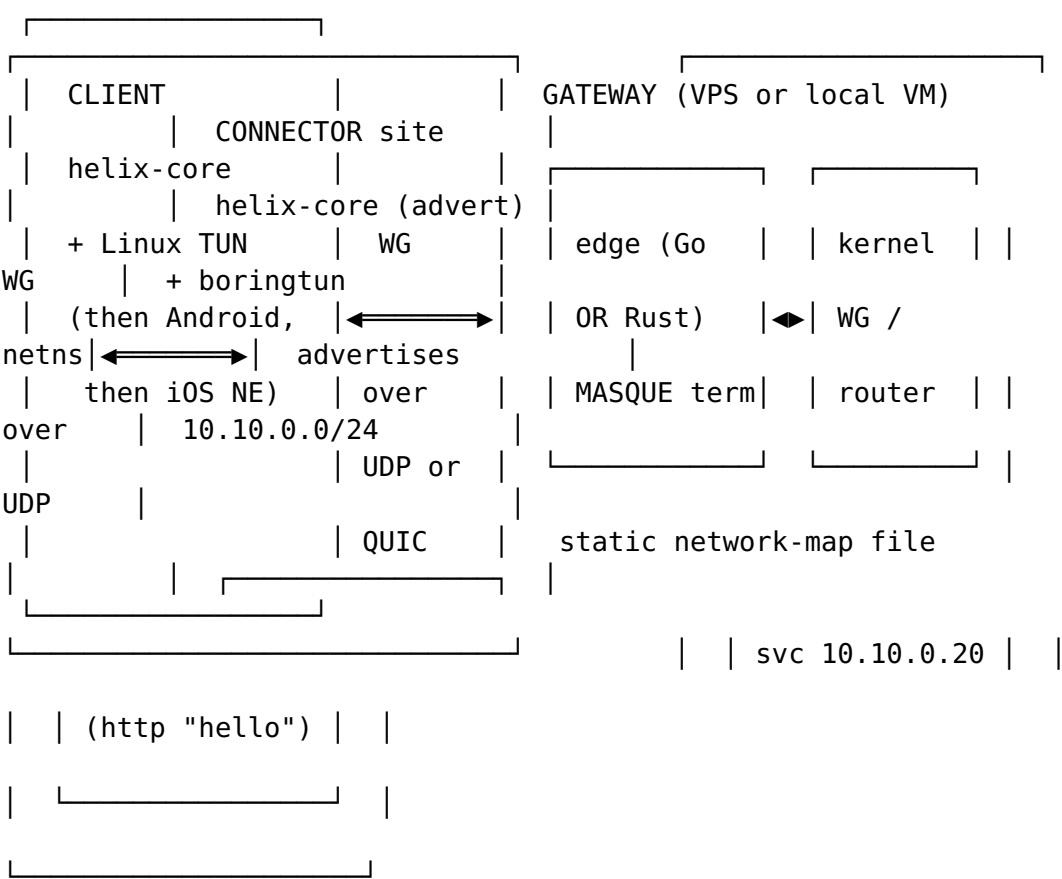
0. The questions Phase 0 must answer (exit gates)

#	Question	Gate (go / no-go)	Section
G1	Can helix-core (Rust + boringtun) move real traffic through the gateway to a connector's LAN over plain UDP ?	iperf3 through-tunnel $\geq$ 80% of bare-link throughput; ping to LAN host succeeds	§4, §8
G2	Can the same core do it over MASQUE/QUIC (WG-in-HTTP/3)?	Works through a DPI-style UDP block; $\geq$ 50% of plain-UDP throughput; survives 5% packet loss better than UoT	§5, §8
G3	Does the Rust core run inside an iOS NEPacketTunnelProvider under its memory budget with headroom?	Steady-state RSS stays under the device-enforced ceiling with $\geq$ 30% headroom during a 1 GB transfer	§6
G4	Go vs Rust for the gateway edge — which terminates MASQUE better, all-in?	Decision recorded with benchmark numbers + reuse/velocity assessment	§7
G5	Is the flutter_rust_bridge FFI boundary clean enough to drive the core from Dart (connect/disconnect/status stream)?	Flutter Linux app toggles the tunnel and shows live status via the core's event stream	§9
G6	Does push-based reconciliation work at all (static network-map $\rightarrow$ core brings up peers)?	Core consumes a map document and converges without restart	§10

A "no-go" on G3 or G4 changes the architecture, not just the schedule — that's exactly why they're in Phase 0.

1. The vertical slice (what we build, precisely)

One client, one gateway, one connector, one LAN host. Nothing else.



In scope: WG tunnels both sides; plain-UDP and MASQUE transports; static network map (a YAML/JSON file, no Go control plane yet); a "reach 10.10.0.20:80 and get a hello page" success test; the FFI + one Flutter toggle; the iOS memory harness; the dual edge benchmark.

Explicitly out of scope for Phase 0: Postgres/Redis, the real control plane, policy engine, multi-tenant, Console UI beyond a toggle, Shadowsocks/UoT/LWO/DAITA, multi-hop, HarmonyOS/Aurora, key rotation, billing. Those are Phase 1+. Faking them with static files is the whole point.

2. Spike milestones (each independently demoable)

ID	Milestone	Proves	Days (rough)
S0	Cargo workspace + helix-transport trait + plain-UDP impl; loopback echo test	the abstraction compiles and round-trips	2
S1	helix-core wraps boringtun; Linux client ↔ Linux gateway plain-UDP WG up; ping overlay IP	WG core works through our transport	3
S2	Connector advertises 10.10.0.0/24 (netns LAN); client reaches 10.10.0.20:80	the two-way routing slice (G1)	3
S3	MASQUE transport (quinn): same slice over :443/udp HTTP/3; works through a UDP-block rule	the QUIC headline (G2)	5
S4	Go edge (quic-go + masque-go) terminating the same MASQUE; A/B benchmark vs Rust edge	the edge decision (G4)	4
S5	flutter_rust_bridge FFI; Flutter-Linux app: connect/disconnect + live status stream	the FFI boundary (G5)	3
S6	Android VpnService + JNI loads helix-core; tunnel up on a phone	mobile path real	3
S7	iOS NEPacketTunnelProvider + Rust static lib; memory harness under 1 GB transfer	the make-or-break gate (G3)	4
S8	Static network-map ingest + reconcile (bring peer up from a map delta, no restart)	the real-time model (G6)	2

S3, S4, S7 are the high-risk ones; sequence them so a failure surfaces early. S0-S2 are the foundation everything else stands on.

3. Test rig & topology (reproducible, no special hardware)

Use **Linux network namespaces** to simulate the connector's private LAN and a "remote service," so the entire slice runs on one Linux box (or two cheap VMs) and is fully scriptable in CI later.

```
# --- connector-side "private LAN" simulated with a netns ---
sudo ip netns add lanA
sudo ip link add veth-h type veth peer name veth-lanA
sudo ip link set veth-lanA netns lanA
sudo ip addr add 10.10.0.1/24 dev veth-h                # connector
               host side
sudo ip -n lanA addr add 10.10.0.20/24 dev veth-lanA # the
               "service" host
sudo ip link set veth-h up
sudo ip -n lanA link set veth-lanA up
sudo ip -n lanA route add default via 10.10.0.1
# run a trivial service the client will reach through the whole
# chain:
sudo ip netns exec lanA python3 -m http.server 80 --bind
    10.10.0.20 &

# --- DPI/censorship simulation for G2: block plain WireGuard UDP,
#      allow 443 ---
# (apply on the path between client and gateway to force MASQUE)
sudo nft add table inet dpi
sudo nft add chain inet dpi fwd '{ type filter hook forward
    priority 0; }'
sudo nft add rule inet dpi fwd udp dport 51820 drop      # kill
    plain WG
sudo nft add rule inet dpi fwd udp dport 443 accept      # allow
    QUIC/443

# --- impairment for loss-resilience tests (G2) ---
sudo tc qdisc add dev <iface> root netem loss 5% delay 40ms 10ms
```

Three processes total: client (helix-core), gateway (edge + kernel WG or boringtun + the static router into lanA), connector (helix-core advertising 10.10.0.0/24, sitting on veth-h). Success = curl from the client's overlay namespace to http://10.10.0.20/ returns the hello page, with plain-UDP **and** with plain-UDP blocked (MASQUE only).

4. helix-core — the Rust workspace (S0-S2)

The interfaces below are the ones meant to **survive into Phase 1**. Keep them stable; let the bodies be scrappy.

4.1 Workspace layout

```
helix-core/
├─ Cargo.toml                # [workspace]
├─ crates/
│   └─ helix-transport/      # the pluggable obfuscation layer
    (shared client+edge+connector)
│   │   └─ src/lib.rs        # Transport trait + registry
│   │   └─ src/plain_udp.rs  # S0/S1
│   │   └─ src/masque.rs     # S3 (quinn + HTTP/3 datagrams)
│   └─ helix-wg/            # boringtun wrapper: handshake,
    encrypt/decrypt, timers
│   └─ helix-tun/           # OS tun device abstraction (Linux
    now; shims call in later)
│   └─ helix-core/          # orchestrator: ties tun <-> wg <-
> transport; event stream
│   └─ helix-ffi/           # flutter_rust_bridge surface (S5)
└─ bin/
    └─ helix-client.rs       # Linux CLI client (S1/S2)
    └─ helix-connector.rs    # Linux CLI connector (S2)
```

4.2 The transport trait (the single most important interface in the project)

Everything obfuscation-related hides behind this. `plain_udp` and `masque` implement it; the client, connector, and gateway edge all consume it. **One trait, three consumers, N transports** — this is the "single implementation" guarantee from the architecture doc made concrete.

```
// helix-transport/src/lib.rs
use async_trait::async_trait;
use bytes::Bytes;
use std::net::SocketAddr;

/// A bidirectional carrier for already-encrypted WireGuard
    datagrams.
/// The transport NEVER sees plaintext; it only changes how WG
    bytes look on the wire.
#[async_trait]
pub trait Transport: Send + Sync {
    /// Send one WG datagram toward the peer endpoint.
    async fn send(&self, datagram: Bytes) -> Result<,
        TransportError>;
```

```

/// Receive the next WG datagram from the peer (cancel-safe).
async fn recv(&self) -> Result<Bytes, TransportError>;

/// Human label for logs/metrics: "plain-udp", "masque-h3",
"shadowsocks", ...
fn kind(&self) -> &'static str;

/// MTU the upper layer (WG) may use over this transport,
after overhead.
fn effective_mtu(&self) -> u16;
}

#[derive(Clone, Debug)]
pub enum TransportConfig {
    PlainUdp { peer: SocketAddr, bind: SocketAddr },
    Masque { url: String, sni: String, bind: SocketAddr }, //
        https://gw:443
    // Phase 1+: Shadowsocks { .. }, UdpOverTcp { .. }, Lwo { .. }
}

/// Build a transport from config. The escalation ladder (auto
mode) just
/// constructs the next variant on repeated handshake failure.
pub async fn dial(cfg: TransportConfig) -> Result<Box<dyn
    Transport>, TransportError> { /* ... */ }

```

Design note: WG datagrams are independent and loss-tolerant, so the transport carries **unreliable datagrams**, never an ordered byte stream. For MASQUE that means QUIC DATAGRAM frames (RFC 9221) via HTTP Datagrams, *not* a QUIC stream — this preserves WG's own loss semantics and avoids head-of-line blocking. Getting this right is half the point of S3.

4.3 Plain-UDP transport (S0/S1)

Trivial, but it validates the trait and gives the throughput **baseline** every other transport is measured against.

```

// helix-transport/src/plain_udp.rs
pub struct PlainUdp { sock: tokio::net::UdpSocket, peer:
    SocketAddr }

#[async_trait]
impl Transport for PlainUdp {
    async fn send(&self, dg: Bytes) -> Result<(), TransportError>
    {
        self.sock.send_to(&dg, self.peer).await?; Ok(())
    }
    async fn recv(&self) -> Result<Bytes, TransportError> {
        let mut buf = vec![0u8; 1500];
        let n = self.sock.recv(&mut buf).await?;
    }
}

```

```

        buf.truncate(n); Ok(Bytes::from(buf))
    }
    fn kind(&self) -> &'static str { "plain-udp" }
    fn effective_mtu(&self) -> u16 { 1420 } // standard WG-over-
        IPv4
}

```

4.4 helix-wg (boringtun wrapper)

boringtun is the right Phase-0 choice: pure-Rust userspace WireGuard, no kernel module, runs identically on Linux/Android/iOS — which is exactly the iOS path (no kernel WG on iOS regardless). Wrap its Tunn state machine; pump its timer; route its four output verdicts.

```

// helix-wg/src/lib.rs (sketch)
pub struct WgPeer { tunn: boringtun::noise::Tunn, /* keys,
    endpoint */ }

impl WgPeer {
    /// Inbound: bytes arrived from the transport -> maybe
    /// plaintext for the TUN,
    /// or a handshake reply to send back out the transport.
    pub fn handle_transport_in(&mut self, datagram: &[u8],
        scratch: &mut [u8]) -> WgVerdict { /* ... */ }
    /// Outbound: a plaintext IP packet from the TUN -> encrypted
    /// datagram for transport.
    pub fn handle_tun_out(&mut self, ip_pkt: &[u8], scratch: &mut
        [u8]) -> WgVerdict { /* ... */ }
    /// Call ~every 100-250ms: emits keepalives / rekeys.
    pub fn tick(&mut self, scratch: &mut [u8]) -> WgVerdict { /
        * ... */ }
}
// WgVerdict = WriteToTransport(Bytes) | WriteToTun(Bytes) |
    Nothing | Err

```

4.5 helix-core orchestrator + the connector mode

The orchestrator runs three loops: tun → wg → transport, transport → wg → tun, and timer tick. The **connector** is the *same core* with a different routing posture: instead of capturing the device's default route, it (a) advertises 10.10.0.0/24 in its map, and (b) NATs/ forwards decapsulated packets into veth-h and back. One binary, a --mode={client|connector} flag in Phase 0.

```

client:    [app traffic] → TUN → WG encrypt → Transport →
gateway
connector: gateway → Transport → WG decrypt → forward into
10.10.0.0/24 (and reverse)

```


The orchestrator also exposes an **event stream** (`tokio::sync::broadcast`) — Connecting | Handshaking | Connected{transport, rtt} | Reconnecting | Down{reason} — which is what the FFI surfaces to Flutter in S5 and what proves the "real-time status" UX.

5. MASQUE / QUIC transport (S3 — the headline, and the riskiest Rust bet)

This is where Phase 0 earns its keep. The goal: wrap each WireGuard datagram in an HTTP/3 **CONNECT-UDP** exchange (RFC 9298) so the flow on `:443/udp` is indistinguishable from a browser doing HTTP/3 — the exact mechanism Mullvad ships.

5.1 How the bytes flow

```
client WG datagram (Bytes)
  ↳ HTTP Datagram (RFC 9297, context-id = 0 for UDP payload)
    ↳ QUIC DATAGRAM frame (RFC 9221, unreliable – matches WG semantics)
      ↳ QUIC/HTTP-3 connection to https://gateway:443 (looks like web)
        ↳ edge: extract HTTP Datagram → WG datagram → kernel WG
```

The CONNECT-UDP request establishes the proxied UDP "flow" to the gateway's internal WG socket; thereafter WG datagrams ride as HTTP Datagrams over QUIC DATAGRAM frames. No per-packet HTTP round trip — the tunnel is set up once, then it's pure datagram relay.

5.2 Rust building blocks (and the maturity caveat)

- quinn — mature QUIC. Exposes unreliable datagrams (`Connection::send_datagram/read_datagram`). Solid.
- h3 — HTTP/3 on quinn. Exists, less battle-tested than Go's stack.
- **MASQUE capsule/CONNECT-UDP handling** — in Rust this is *thin-to-absent* as a turnkey crate; expect to implement the CONNECT-UDP request + HTTP-Datagram framing by hand on top of h3 + quinn datagrams. **This relative immaturity vs Go's masque-go is itself a Phase-0 finding and a direct input to G4 (§7).**

```
// helix-transport/src/masque.rs (sketch – the parts that matter)
pub struct MasqueTransport {
    conn: quinn::Connection, // established QUIC/H3
    connection to gw:443
```

```

    // flow context for CONNECT-UDP established at dial()
}

#[async_trait]
impl Transport for MasqueTransport {
    async fn send(&self, wg: Bytes) -> Result<(), TransportError>
    {
        let http_dg = encode_http_datagram(/*ctx*/0, &wg); // RFC
        9297 framing

        self.conn.send_datagram(http_dg)?; // RFC
        9221 QUIC datagram
        Ok(())
    }
    async fn recv(&self) -> Result<Bytes, TransportError> {
        let dg = self.conn.read_datagram().await?;
        Ok(decode_http_datagram(dg)?) //
        strip framing -> WG bytes
    }
    fn kind(&self) -> &'static str { "masque-h3" }
    fn effective_mtu(&self) -> u16 { 1280 } // QUIC overhead eats
        headroom; measure & tune
}

```

5.3 What S3 must demonstrate

1. Slice works end-to-end with **plain WG UDP blocked** (the nft rule from §3) — proving real censorship evasion, not just "QUIC also works."
2. The connection **looks like HTTP/3** to a passive observer (verify with a packet capture: QUIC long/short headers, no WG signature; ideally TLS SNI matching the masquerade host).
3. **Loss resilience:** under netem loss 5%, MASQUE/QUIC sustains higher goodput than a UDP-over-TCP strawman — QUIC's loss recovery is the reason mobile got this feature.
4. **Overhead is quantified:** record the MTU/throughput/CPU penalty vs plain UDP so the auto-ladder's cost model is real.

5.4 Masquerade (carry-forward, native)

The edge's :443 should serve a believable decoy site to anything that *isn't* a valid CONNECT-UDP flow (probes, scanners). Phase 0: a static "it's just a website" page behind the same QUIC listener. This replaces the original doc's Nginx-camouflage trick with a native edge behavior.

6. iOS memory-ceiling experiment (S7 — the make-or-break gate, G3)

If the core can't live inside a `NEPacketTunnelProvider`, the whole "Rust core on every platform" thesis is wounded — so we test it *early-ish* and *honestly*.

6.1 The constraint

A Network Extension's packet-tunnel process runs with a **far tighter memory budget than the host app** — historically cited around ~15 MB, raised on newer iOS for packet tunnel providers but still strict and silently fatal: exceed it and iOS kills the extension, dropping the tunnel. The exact ceiling is device/OS-dependent and **must be measured, not assumed**. This tight budget is the single strongest reason the data plane is Rust (no GC, bounded allocations) rather than Go.

6.2 Harness

```
ios-spike/  
├─ HelixTunnel/                                # NEPacketTunnelProvider (Swift)  
│   └─ PacketTunnelProvider.swift  
├─ libhelix_core.a                             # Rust staticlib, aarch64-apple-  
ios, --release + LTO + strip  
└─ helix_core.h                               # cbindgen header (or UniFFI-  
generated)
```

```
// PacketTunnelProvider.swift (skeleton)  
import NetworkExtension  
class PacketTunnelProvider: NEPacketTunnelProvider {  
    override func startTunnel(options: [String:NSObject]?,  
                               completionHandler: @escaping (Error?)->Void) {  
        // 1. configure NEPacketTunnelNetworkSettings (overlay IP,  
        //      routes, DNS)  
        // 2. helix_core_start(config) // Rust core takes over  
        //      packetFlow read/write  
        // 3. pump: packetFlow.readPackets -> helix_core_tun_out;  
        //      core -> packetFlow.writePackets  
    }  
}
```

6.3 Method (be rigorous — this number decides architecture)

- Build the **same helix-core** as aarch64-apple-ios staticlib, opt-level=z/s + LTO + panic=abort + strip; record binary size.
- Drive a **sustained 1 GB transfer** through the tunnel on a **real device** (Simulator memory behavior is not representative).

- Sample the **extension process** RSS via Xcode Instruments (Allocations + VM Tracker), not the host app. Record peak and steady-state.
- Run **plain-UDP** and **MASQUE** separately — QUIC buffers cost more memory; both must pass.
- **Pass:** steady-state peak stays under the device-enforced ceiling with **≥ 30% headroom** for both transports across a 30-minute soak.

6.4 If it fails

Documented fallbacks, in order: (1) shrink buffers / cap QUIC flow-control windows in the iOS build; (2) move MASQUE off-device for iOS (client uses plain WG + on-path obfuscation only) — partial feature loss; (3) split the core so only the lean WG datapath is in-extension and QUIC negotiation lives in the app via app-extension IPC. Each fallback is a real product decision, which is why we surface it in Phase 0.

7. Go vs Rust gateway edge (S4 — decision G4)

The architecture doc *recommends* Rust (single MASQUE implementation shared with clients) but flags Go as acceptable. Phase 0 **builds both** for the MASQUE termination path and decides on numbers.

7.1 The two contenders (identical job: terminate MASQUE on :443, hand WG to kernel)

	Rust edge (helix-edge)	Go edge
QUIC/H3	quinn + h3 (+ hand-rolled CONNECT-UDP)	quic-go + masque-go (turnkey CONNECT-UDP/IP)
Code reuse	Shares helix-transport byte-for-byte with clients	Separate implementation; must track the Rust one
Fits stack	new language in the server tree	matches the Go/Gin control plane exactly
Hot-path cost	no GC	GC pauses under load (measure)
MASQUE maturity	thinner ecosystem (a Phase-0 risk)	more mature MASQUE tooling

7.2 Benchmark protocol (run both, same rig, same kernel-WG backend)

- **Throughput:** iperf3 through-tunnel, 1 / 10 / 100 concurrent clients.
- **CPU per Gbps** at the edge (the cost-to-serve number).
- **Latency:** p50/p99 RTT added by the edge; for Go, watch GC-induced p99 tail.
- **Connection churn:** N handshakes/sec the edge sustains (mobile roaming = constant re-handshakes).
- **Memory under churn.**
- **Dev cost (qualitative):** hours to get CONNECT-UDP correct in each; lines of bespoke code.

7.3 Decision matrix (fill from S4 results)

Criterion	Weight	Rust	Go
Reuse / single-impl guarantee	high		✗
MASQUE implementation effort	high	?	
Throughput & CPU/Gbps	high	?	?
p99 latency under load	med	?	?
Fits existing Go control plane	med	✗	
Team velocity	med	?	?

Default lean: if Rust gets within ~10–15% on throughput/CPU and the hand-rolled CONNECT-UDP isn't a quagmire, choose **Rust** for the single-implementation win. If MASQUE-in-Rust proves painful or Go wins decisively on cost-to-serve, choose **Go** and accept dual MASQUE implementations (mitigated by sharing test vectors + a conformance suite across both). Record the call in the decision log (§12).

8. Measurement methodology (applies to G1, G2, G4)

One harness, run for every transport × edge combination so results are comparable.

Metric	How	Pass bar
Through-tunnel throughput	iperf3 -c <lan-host-via-tunnel> (TCP + UDP)	plain-UDP ≥ 80% bare link; MASQUE ≥ 50% of plain-UDP
Added latency	ping overlay vs bare; p50/p99	

Metric	How	Pass bar
		plain-UDP < 2 ms added; MASQUE < 15 ms added
Loss resilience	tc netem loss 5% delay 40ms; compare goodput	MASQUE/QUIC > UoT strawman
Handshake time	timestamp connect→first-data	< 1 s plain; < 2 s MASQUE
Reconnect/roam	flap the client iface; time to recovery	< 3 s
Edge CPU per Gbps	pidstat/perf at saturation	record (cost-to-serve)
Core RSS (per platform)	/proc (Linux), Instruments (iOS), dumsys meminfo (Android)	iOS gate per §6.3
Wire fingerprint	tshark capture of MASQUE flow	classified as HTTP/3, no WG signature

Script it (bench.sh) so S4's A/B and S7's transport pair run identically and land in a CSV → the decision tables fill themselves.

9. FFI boundary — Flutter ↔ Rust core (S5, gate G5)

Use **flutter_rust_bridge v2**: generates the Dart↔Rust glue, handles async, and streams. The surface stays tiny and stable.

```
// helix-ffi/src/api.rs (flutter_rust_bridge generates Dart
// bindings from this)
pub struct ClientConfig { pub map_path: String, pub transport:
    String /* auto|plain|masque */ }

pub async fn start(cfg: ClientConfig) -> anyhow::Result<> { /*
    spin up helix-core orchestrator */ }
pub async fn stop() -> anyhow::Result<> { /* tear down */ }

/// Live status as a stream the UI subscribes to (maps to the
/// broadcast channel in §4.5).
pub fn status_stream(sink: StreamSink<TunnelStatus>) { /* forward
    core events */ }

#[frb(mirror)]
pub enum TunnelStatus {
    Connecting, Handshaking,
    Connected { transport: String, rtt_ms: u32 },
```

```

    Reconnecting, Down { reason: String },
}

// Flutter (app_access) – the entire happy path the spike must
// show:
await HelixCore.start(ClientConfig(mapPath: '/etc/helix/
    map.json', transport: 'auto'));
HelixCore.statusStream().listen((s) => setState(() => status =
    s)); // live chip updates
// ...toggle off:
await HelixCore.stop();

```

G5 pass: a Flutter-Linux window with one connect/disconnect toggle and a status chip that goes Connecting → Handshaking → Connected (masque, 23ms) driven *only* by the Rust event stream. On Linux the core can drive the TUN directly; on mobile the platform shim (S6/S7) owns the TUN and the same FFI drives logic.

10. Static network-map + reconciliation (S8, gate G6)

No Go control plane yet — fake it with a file that has the *exact shape* Phase 1's WatchNetworkMap will stream, so the reconciler is real even though the source is static.

```

// /etc/helix/map.json – the desired-state document the core
// reconciles to
{
  "self": { "overlay_ip": "fd7a:helix:1::2/128", "transport":
    "auto" },
  "gateway": { "endpoint": "gw.example:443", "wg_pubkey": "...",
    "masque_sni": "cdn.example" },
  "peers": [ { "name": "connectorA", "wg_pubkey": "...",
    "allowed_ips": ["10.10.0.0/24"] } ],
  "dns": [ "fd7a:helix:1::1" ]
}

```

The reconciler diffs desired-vs-actual and converges (add/remove peers, switch transport) **without restarting** the process. **G6 pass:** edit map.json to add the connector's prefix → the core picks up the delta (file-watch standing in for a stream event) and the client can now reach 10.10.0.20 — no restart, no reconnect of unrelated state. This is the literal seed of the Phase-1 push model.

11. Deliverables (definition of done for Phase 0)

1. helix-core Rust workspace: transport trait + plain-UDP + MASQUE, boringtun WG, TUN, orchestrator, client+connector binaries.
2. Both edges (helix-edge Rust + Go edge) terminating MASQUE, with the bench.sh CSV comparing them.
3. Reproducible netns rig + DPI-block + netem scripts (the §3 + §8 harness), runnable in one make spike.
4. Flutter-Linux app proving the FFI toggle + live status (G5).
5. Android build with the tunnel up on a real phone (S6).
6. iOS NE build + the **memory report** (peak/steady RSS, both transports, 30-min soak) — the G3 verdict.
7. **Decision log** (§12) with G1-G6 outcomes and the edge-language call.
8. A 5-minute **demo script**: plain-UDP slice → block WG → MASQUE slice survives → flip a map entry → new network reachable, all narrated.

12. Decision log (fill as gates clear)

Gate	Outcome	Evidence (numbers / capture)	Decision / consequence
G1 plain-UDP slice	☐ pass ☐ fail	iperf ..., ping ...	—
G2 MASQUE through DPI block	☐ pass ☐ fail	goodput @5% loss ..., tshark ...	—
G3 iOS memory	☐ pass ☐ fail	peak RSS ... / ceiling ... / headroom ... %	core stays Rust ↔ fallback §6.4
G4 edge language	☐ Rust ☐ Go	CPU/Gbps R:... G:...; p99 R:... G:...; MASQUE effort ...	single-impl ↔ dual-impl
G5 FFI	☐ pass ☐ fail	demo recording	frb v2 ↔ UniFFI
G6 reconcile	☐ pass ☐ fail	map-delta → reachable, no restart	confirms push model

13. Risks specific to Phase 0 (and pre-planned fallbacks)

Risk	Likelihood	Fallback
MASQUE/ CONNECT-UDP in Rust takes longer than budget	med-high	Prototype MASQUE first in Go (masque-go) to unblock G2, then port — and let that inform G4 toward Go
boringtun quirks / maintenance gaps	med	Swap to wireguard-go via cgo for the spike, or kernel WG on Linux for non-iOS gates
iOS NE memory fails	med	§6.4 ladder; worst case iOS ships plain-WG + on-path obfs only in v1
h3 immaturity blocks datagrams	med	Pin versions; if blocked, carry WG over a QUIC <i>stream</i> as a stopgap (accept HoL blocking) purely to pass G2, flag as non-final
Time-box blown	—	Stop; the unfinished gate is the finding. Escalate the architectural decision with partial data rather than overrun silently

14. How Phase 0 feeds Phase 1

Every surviving interface graduates: the Transport trait gains Shadowsocks/UoT/LWO impls; the static `map.json` shape becomes the `WatchNetworkMap` protobuf streamed by the Go coordinator; the FFI surface gains policy/multi-hop fields; the winning edge becomes `helix-edge` in the monorepo (§11 of the architecture doc); the bench harness becomes CI gates with the §8 bars as thresholds. Nothing built here is wasted *if the interfaces hold* — which is the entire discipline of Phase 0.

End of Phase 0 spike plan. Pair with `HelixVPN-Architecture-Refined.md` §12 (roadmap) — clearing G1-G6 is the entry condition for Phase 1 (MVP self-host).